

RELAZIONE: Errori di arrotondamento e stabilità numerica

Anno accademico **2024/25**

Gruppo: Jeffrey Germano

Matricola di riferimento: **5669424**

Obiettivi di questo laboratorio:

Lo scopo dell'esercitazione è studiare alcuni effetti fondamentali dell'aritmetica di macchina e dell'errore numerico, mettendo in relazione i risultati ottenuti con il calcolatore con la teoria del corso. In particolare, si analizzano la non associatività della somma in doppia precisione, l'approssimazione della funzione esponenziale mediante polinomi di Taylor e il calcolo della precisione di macchina nei formati float e double. L'obiettivo è evidenziare come errori di arrotondamento, cancellazione numerica e scelta dell'algoritmo possano influenzare in modo significativo la qualità del risultato finale.

Richiami teorici essenziali:

L'errore assoluto nella stima di una quantità x mediante un valore approssimato $\tilde{x}$ è definito come $|\tilde{x} - x|$. L'errore relativo, quando $x \neq 0$, è invece dato da

$$\frac{|\tilde{x} - x|}{|x|}.$$

Queste due misure permettono di valutare quanto un risultato numerico si discosti dal valore esatto, in termini sia assoluti sia proporzionali alla grandezza del dato.

In aritmetica floating-point, ogni operazione introduce un errore di arrotondamento, che in prima approssimazione è limitato da una costante di macchina dell'ordine della precisione del formato utilizzato. Per questo motivo, risultati matematicamente equivalenti possono produrre esiti numericamente diversi, a seconda dell'ordine delle operazioni e della forma con cui vengono calcolati.

Un fenomeno particolarmente importante è la **cancellazione numerica**, che si verifica quando si sommano o sottraggono numeri quasi opposti: in questo caso una parte significativa delle cifre corrette può andare perduta e l'errore relativo può essere fortemente amplificato. Negli appunti questo comportamento viene descritto tramite i coefficienti di amplificazione della somma, che diventano molto grandi quando i termini sono quasi discordi.

Si dice infine che un algoritmo è **stabile** se gli errori numerici che introduce restano dello stesso ordine di grandezza di quelli inevitabili dovuti alla rappresentazione floating-point dei dati. In generale, è quindi preferibile adottare formulazioni che evitino cancellazioni pesanti o che riorganizzino i calcoli in modo numericamente più favorevole. Questo aspetto sarà centrale soprattutto nel confronto tra i due algoritmi per l'approssimazione di e^x .

Descrizione del codice:

Il programma è organizzato in tre funzioni principali, una per ciascun esercizio, richiamate tramite un semplice menù nel main, così da permettere l'esecuzione separata dei test richiesti dalla traccia.

Nel **primo esercizio** si analizza la non associatività della somma in doppia precisione confrontando i valori di $(a + b) + c$ e $a + (b + c)$ per i diversi valori di a , costruiti a partire dalla matricola, con b molto grande e $c = -b$.

Nel **secondo esercizio** si approssima la funzione esponenziale e^x mediante il polinomio di Taylor di grado N , confrontando due algoritmi diversi e calcolando, per ciascun caso, l'errore assoluto e relativo rispetto a $\exp(x)$.

Nel **terzo esercizio** si determina la precisione di macchina per i tipi float e double, confrontando il valore ottenuto numericamente con quello restituito da `std::numeric_limits<...>::epsilon()`.

Per rendere più chiaro il confronto tra i risultati, il programma stampa tutti i valori in notazione scientifica con precisione elevata, così da visualizzare correttamente sia i numeri molto piccoli sia quelli molto grandi e da confrontare in modo affidabile errori e differenze tra algoritmi.

Esercizio 1 – Non associatività della somma

Impostazione

Con matricola 5669424 si ha $d_0 = 4$ (ultima cifra) e $d_1 = 2$ (penultima cifra). Si considerano quindi i valori di a richiesti dalla traccia e si confrontano in doppia precisione le due associazioni della somma:

$$(a + b) + c \quad \text{e} \quad a + (b + c),$$

con b molto grande e $c = -b$.

Commento ai risultati ottenuti

In aritmetica esatta le due espressioni dovrebbero fornire lo stesso risultato, perché la somma è associativa. In aritmetica floating-point, invece, il risultato dipende dall'ordine con cui vengono eseguite le operazioni, poiché ogni somma è soggetta ad arrotondamento.

Nel caso di $a + (b + c)$, la quantità $(b + c)$ viene calcolata per prima e risulta esattamente nulla, perché c è definito come l'opposto di b nello stesso formato floating-point; di conseguenza il risultato finale coincide esattamente con a . Nel caso di $(a + b) + c$, invece, la somma iniziale tra a e b può perdere completamente o parzialmente il contributo di a , poiché b è molto più grande e il numero risultante viene arrotondato al valore rappresentabile più vicino. Quando poi si aggiunge $c = -b$, l'informazione persa non può più essere recuperata.

L'output conferma chiaramente questo fenomeno. Per i valori più piccoli di a , il contributo di a viene completamente assorbito e il risultato di $(a + b) + c$ è zero, mentre $a + (b + c)$ restituisce correttamente a . Per valori più grandi di a , si osservano risultati come 65536, 524288 e 4980736, che riflettono il passo di quantizzazione dei numeri double nella zona dei numeri molto grandi. L'esercizio mostra quindi in modo diretto che la somma, in aritmetica di macchina, non è associativa.

Esercizio 2 – Taylor di e^x : confronto tra 2 algoritmi

Impostazione

Si considera il polinomio di Taylor di grado N per l'approssimazione di e^x , confrontando il valore approssimato con $\exp(x)$ tramite errore assoluto ed errore relativo. I test vengono eseguiti per

$$x = 0.5, 30, -0.5, -30$$

e per i valori

$$N = 3, 10, 50, 100, 150.$$

Si confrontano due strategie di calcolo:

- **Algoritmo 1:** calcolo diretto del polinomio di Taylor di e^x ;
- **Algoritmo 2:** per valori negativi, uso dell'identità

$$e^{-x} = \frac{1}{e^x}$$

calcolando prima $e^{|x|}$ e poi il reciproco.

Commento ai risultati ottenuti

Algoritmo 1 (Taylor diretto)

Per $x = 0.5$ e $x = -0.5$, il polinomio di Taylor converge rapidamente e l'approssimazione diventa molto accurata già per valori moderati di N . In questi casi l'errore assoluto e relativo decrescono fino a raggiungere i limiti della precisione floating-point.

Per $x = 30$, il comportamento è diverso: con valori piccoli di N il polinomio troncato è ancora lontano dal valore esatto, quindi l'errore è dominato dal troncamento. Aumentando N , l'approssimazione migliora progressivamente fino a diventare molto accurata.

Il caso numericamente più delicato è $x = -30$. Con l'**algoritmo 1** (Taylor diretto), i termini della serie sono molto grandi in valore assoluto e di segno alternato, mentre il risultato finale e^{-30} è piccolissimo. Questo provoca una forte cancellazione numerica e una grande perdita di cifre significative: di conseguenza, anche aumentando N , l'approssimazione diretta rimane molto lontana dal valore corretto.

L'**algoritmo 2** evita invece questa cancellazione, perché calcola prima e^{30} , che è un numero grande ma ben rappresentabile, e solo alla fine ne prende il reciproco. Per questo motivo, nel caso $x = -30$, il secondo algoritmo fornisce risultati molto più accurati e stabili. L'esercizio mostra quindi chiaramente che due procedimenti matematicamente equivalenti possono avere un comportamento numerico molto diverso, e che una riformulazione opportuna del problema può migliorare in modo decisivo la stabilità del calcolo.

Esercizio 3 – Precisione di macchina (epsilon)

Impostazione

Si vuole determinare la precisione di macchina nei formati `float` e `double`, cioè il più piccolo numero positivo ε tale che, in aritmetica floating-point, valga ancora

$$1 + \varepsilon > 1.$$

Nel programma questo valore viene ricercato nella forma

$$\varepsilon = 2^{-d},$$

determinando il massimo intero d per cui la condizione precedente è ancora soddisfatta.

Commento ai risultati ottenuti

I risultati ottenuti sono pienamente coerenti con la teoria. Per il tipo `float` si trova $d = 23$, mentre per il tipo `double` si trova $d = 52$. Di conseguenza, gli epsilon calcolati coincidono con quelli restituiti da `std::numeric_limits<...>::epsilon()`, confermando la correttezza dell'algoritmo implementato.

Questo risultato riflette direttamente il numero di bit effettivi della mantissa nei due formati: la precisione disponibile in `double` è molto maggiore di quella in `float`, e questo spiega perché gli errori di arrotondamento siano generalmente più piccoli quando si lavora in doppia precisione. L'esercizio fornisce quindi una verifica concreta del significato numerico della precisione di macchina e del suo legame con la rappresentazione floating-point dei numeri.

Conclusioni

L'esercitazione mette in evidenza alcuni aspetti fondamentali dell'aritmetica di macchina e del calcolo numerico. Il primo esercizio mostra che, in floating-point, l'ordine delle operazioni può modificare il risultato, evidenziando in modo concreto la non associatività della somma e la perdita di significato dovuta alla cancellazione numerica. Il secondo esercizio conferma che due algoritmi matematicamente equivalenti possono avere un comportamento numerico molto diverso: nel caso dell'approssimazione di e^x , una riformulazione opportuna del calcolo consente di evitare instabilità e ottenere risultati molto più accurati. Il terzo esercizio, infine, verifica sperimentalmente il significato della precisione di macchina, mettendo in relazione l'epsilon con la rappresentazione floating-point nei formati `float` e `double`.

Nel complesso, la relazione conferma che la qualità di un risultato numerico non dipende solo dalla formula matematica utilizzata, ma anche dal modo in cui essa viene implementata e calcolata in macchina.

Output ottenuto

Di seguito l'output completo ottenuto, utile per documentare i risultati:

```
--- MENU ---
```

1. Esercizio 1 (aritmetica di macchina)
2. Esercizio 2 (Taylor exp e errori)
3. Esercizio 3 (epsilon di macchina)
0. Esci

Scegli un esercizio: 1

```
--- ESERCIZIO 1 -----
```

```
i=0  a=5.000000000000000e+000
(a+b)+c = 0.000000000000000e+000
a+(b+c) = 5.000000000000000e+000
diff    = 5.000000000000000e+000 (rel wrt a) = 1.000000000000000e+000
```

```
i=1  a=5.000000000000000e+001
(a+b)+c = 0.000000000000000e+000
a+(b+c) = 5.000000000000000e+001
diff    = 5.000000000000000e+001 (rel wrt a) = 1.000000000000000e+000
```

```
i=2  a=5.000000000000000e+002
(a+b)+c = 0.000000000000000e+000
a+(b+c) = 5.000000000000000e+002
diff    = 5.000000000000000e+002 (rel wrt a) = 1.000000000000000e+000
```

```
i=3  a=5.000000000000000e+003
(a+b)+c = 0.000000000000000e+000
a+(b+c) = 5.000000000000000e+003
diff    = 5.000000000000000e+003 (rel wrt a) = 1.000000000000000e+000
```

```
i=4  a=5.000000000000000e+004
(a+b)+c = 6.553600000000000e+004
a+(b+c) = 5.000000000000000e+004
diff    = 1.553600000000000e+004 (rel wrt a) = 3.107200000000000e-001
```

```
i=5  a=5.000000000000000e+005
(a+b)+c = 5.242880000000000e+005
a+(b+c) = 5.000000000000000e+005
diff    = 2.428800000000000e+004 (rel wrt a) = 4.857600000000001e-002
```

```
i=6  a=5.000000000000000e+006
(a+b)+c = 4.980736000000000e+006
a+(b+c) = 5.000000000000000e+006
diff    = 1.926400000000000e+004 (rel wrt a) = 3.852800000000000e-003
```

--- MENU ---

1. Esercizio 1 (aritmetica di macchina)
2. Esercizio 2 (Taylor exp e errori)
3. Esercizio 3 (epsilon di macchina)
0. Esci

Scegli un esercizio: 2

--- ESERCIZIO 2 -----

ALGORITMO 1 (Taylor diretto)

```
x = 5.000000000000000e-001

N = 3  approx=1.645833333333333e+000  exact=1.6487212707001282e+000
abs_err=2.8879373667949348e-003  rel_err=1.7516225562908972e-003

N = 10  approx=1.6487212706873655e+000  exact=1.6487212707001282e+000
abs_err=1.2762679801880950e-011  rel_err=7.7409565999359535e-012

N = 50  approx=1.6487212707001278e+000  exact=1.6487212707001282e+000
abs_err=4.4408920985006262e-016  rel_err=2.6935372142162058e-016

N = 100  approx=1.6487212707001278e+000  exact=1.6487212707001282e+000
abs_err=4.4408920985006262e-016  rel_err=2.6935372142162058e-016

N = 150  approx=1.6487212707001278e+000  exact=1.6487212707001282e+000
abs_err = 4.4408920985006262e-016  rel_err=2.6935372142162058e-016
```

x = 3.0000000000000000e+001

N = 3 approx=4.9810000000000000e+003 exact=1.0686474581524463e+013
abs_err=1.0686474576543463e+013 rel_err=9.999999953389684e-001
N = 10 approx=2.3882962385714287e+008 exact=1.0686474581524463e+013
abs_err=1.0686235751900605e+013 rel_err=9.9997765122426152e-001
N = 50 approx=1.0683289873369205e+013 exact=1.0686474581524463e+013
abs_err=3.1847081552578125e+009 rel_err=2.9801298182693123e-004
N = 100 approx=1.0686474581524465e+013 exact=1.0686474581524463e+013
abs_err = 1.9531250000000000e-003 rel_err=1.8276607361015964e-016
N = 150 approx=1.0686474581524465e+013 exact=1.0686474581524463e+013
abs_err = 1.9531250000000000e-003 rel_err=1.8276607361015964e-016

x = -5.0000000000000000e-001

N = 3 approx=6.0416666666666663e-001 exact=6.0653065971263342e-001
abs_err=2.3639930459667946e-003 rel_err=3.8975656186726400e-003
N = 10 approx=6.0653065972437503e-001 exact=6.0653065971263342e-001
abs_err=1.1741607686133193e-011 rel_err=1.9358638344343909e-011
N = 50 approx=6.0653065971263331e-001 exact=6.0653065971263342e-001
abs_err=1.1102230246251565e-016 rel_err=1.8304483159205278e-016
N = 100 approx=6.0653065971263331e-001 exact=6.0653065971263342e-001
abs_err=1.1102230246251565e-016 rel_err=1.8304483159205278e-016
N = 150 approx=6.0653065971263331e-001 exact=6.0653065971263342e-001
abs_err=1.1102230246251565e-016 rel_err=1.8304483159205278e-016

x = -3.0000000000000000e+001

N = 3 approx=-4.0790000000000000e+003 exact=9.3576229688401748e-014
abs_err=4.0790000000000000e+003 rel_err=4.3590129818038280e+016
N = 10 approx=1.2125484957142858e+008 exact=9.3576229688401748e-014
abs_err=1.2125484957142858e+008 rel_err=1.2957868678316438e+021
N = 50 approx=8.7822922927798986e+008 exact=9.3576229688401748e-014
abs_err=8.7822922927798986e+008 rel_err=9.3851743354310571e+021
N = 100 approx=6.1030551474879851e-006 exact=9.3576229688401748e-014
abs_err=6.1030550539117551e-006 rel_err=6.5220142703272372e+007
N = 150 approx=6.1030424788918156e-006 exact=9.3576229688401748e-014
abs_err=6.1030423853155856e-006 rel_err=6.522007320641428e+007

ALGORITMO 2 ($\exp(-x)$ approx $1 / fN(x)$)

$x = -5.0000000000000000e-001$

N = 3 approx=6.0759493670886078e-001 exact=6.0653065971263342e-001
abs_err=1.0642769962273535e-003 rel_err=1.7546961215968778e-003

N = 10 approx=6.0653065971732856e-001 exact=6.0653065971263342e-001
abs_err=4.6951331711397870e-012 rel_err=7.7409659280279124e-012

N = 50 approx=6.0653065971263354e-001 exact=6.0653065971263342e-001
abs_err=1.1102230246251565e-016 rel_err=1.8304483159205278e-016

N = 100 approx=6.0653065971263354e-001 exact=6.0653065971263342e-001
abs_err=1.1102230246251565e-016 rel_err=1.8304483159205278e-016

N = 150 approx=6.0653065971263354e-001 exact=6.0653065971263342e-001
abs_err=1.1102230246251565e-016 rel_err=1.8304483159205278e-016

$x = -3.0000000000000000e+001$

N = 3 approx=2.0076289901626180e-004 exact=9.3576229688401748e-014
abs_err=2.0076289892268556e-004 rel_err=2.1454476162504439e+009

N = 10 approx=4.1870852696152759e-009 exact=9.3576229688401748e-014
abs_err=4.1869916933855879e-009 rel_err=4.4744180304419147e+004

N = 50 approx=9.3604124932784263e-014 exact=9.3576229688401748e-014
abs_err=2.7895244382514776e-017 rel_err=2.9810182003915718e-004

N = 100 approx=9.3576229688401723e-014 exact=9.3576229688401748e-014
abs_err=2.5243548967072378e-029 rel_err=2.6976454438408703e-016

N = 150 approx=9.3576229688401723e-014 exact=9.3576229688401748e-014
abs_err=2.5243548967072378e-029 rel_err=2.6976454438408703e-016

--- MENU ---

1. Esercizio 1 (aritmetica di macchina)
2. Esercizio 2 (Taylor exp e errori)
3. Esercizio 3 (epsilon di macchina)
0. Esci

Scegli un esercizio: 3

--- ESERCIZIO 3 -----

float : eps=1.1920928955078125e-007 d=23 std_epsilon=1.1920928955078125e-007

double: eps=2.2204460492503131e-016 d=52 std_epsilon=2.2204460492503131e-016

Istruzioni di compilazione ed esecuzione

Istruzioni di compilazione ed esecuzione con Dev-C++

- 1) Aprire **Dev-C++**.
 - 2) Selezionare **File** → **Apri** e scegliere il file **esercitazione_Errori.cpp**.
 - 3) Compilare ed eseguire tramite **Execute** → **Compile & Run** (oppure clic sull'icona **Compile & Run**).
 - 4) Il programma mostra un menù: inserire **1**, **2** o **3** per eseguire rispettivamente gli esercizi 1, 2 o 3, quindi premere Invio. Inserire **0** per terminare.
-

Compilazione da riga di comando (g++)

Da terminale, nella cartella del file:

```
g++ -O2 -std=c++11 esercitazione_Errori.cpp -o esercitazione_Errori
./esercitazione_Errori
```

Compilazione ed esecuzione con Visual Studio Code (g++)

Prerequisiti: avere installato un compilatore C++ (g++).

Procedura:

- 1) Aprire **Visual Studio Code**.
- 2) Aprire la cartella che contiene **esercitazione_Errori.cpp** con **File** → **Open Folder...**
- 3) Installare l'estensione **C/C++** (Microsoft) se non è già presente.
- 4) Aprire il file **esercitazione_Errori.cpp**.
- 5) Aprire il terminale integrato: **Terminal** → **New Terminal**.
- 6) Nel terminale, eseguire i comandi:

Windows (PowerShell / CMD):

```
g++ -O2 -std=c++11 esercitazione_Errori.cpp -o esercitazione_Errori.exe
./esercitazione_Errori.exe
```

Linux / macOS:

```
g++ -O2 -std=c++11 esercitazione_Errori.cpp -o esercitazione_Errori
./esercitazione_Errori
```